



UD7_Práctica2 - Funcións.

Índice

UD7_Práctica2 - Funcións.....	1
Documentación e información.....	2
O que hai que entregar.....	2
Funcións.....	3
Exercicios sobre Funcións.....	3



Documentación e información.

A base de datos de jardinería téndela dispoñible no curso da aula virtual, a carón do bloque de exercicios.

Moitos dos exercicios fanse sobre as mesmas BD, polo que podedes aproveitar o que teñades feito duns exercicios para outros.

O que hai que entregar.

Un documento PDF que inclúa as imaxes seguintes:

- **O código de cada exercicio.** En caso de que o código non quepa nunha imaxe:
 - Copia o código no documento.
 - Inclúe unha imaxe tamén onde se vexa a rutina incluída na BD. Debe verse o nome e a cabeceira da rutina.
- **Os casos de proba** de cada exercicio. Isto inclúe unha imaxe por cada posible resultado da rutina.

As imaxes deben ser válidas: *incluír o teu nome nalgues, o comando, o resultado da execución e o Action Output*



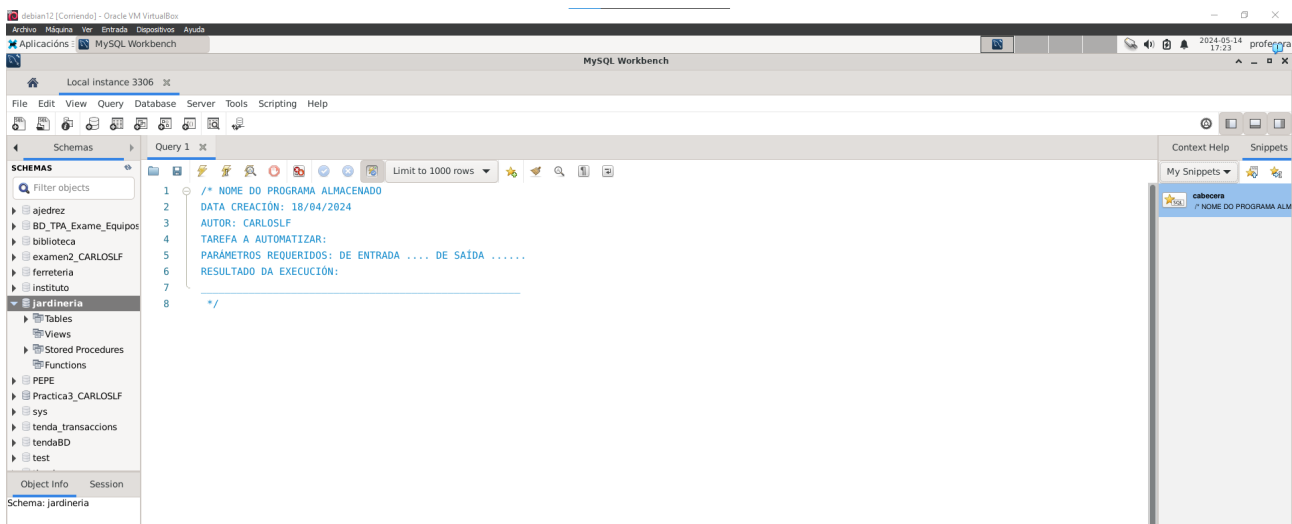
Funcións.

Exercicios sobre Funcións.

Sobre a BD jardinería.

1. Crear un snippets como o do modelo que inclúa o teu nome no campo 'AUTOR'. Engadirase a cada unha das funcións da práctica.

```
1 /*
2 NOME DO PROGRAMA ALMACENADO
3 DATA CREACIÓN:
4 AUTOR:
5 TAREFA A AUTOMATIZAR:
6 PARÁMETROS REQUERIDOS: DE ENTRADA .... DE SAÍDA .....
7 RESULTADO DA EXECUCIÓN:
8 _____
9 */
10 Código procedimento / función / trigger
```



2. Función: teuuser_prezo_total_pedido. Dado un código de pedido a función debe calcular a suma total do pedido. Teña en conta que un pedido pode conter varios produtos diferentes e varias cantidades de cada produto.
 - Parámetros de entrada: codigo_pedido (INT)
 - Parámetros de saída: O prezo total do pedido (FLOAT)
 - Empregar handler para excepcións e warnings.



The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying a SQL script to create a function named `CARLOSFL_prezo_total_pedido`. The script includes a `DELIMITER //` statement, a `CREATE FUNCTION` statement with `RETURNS FLOAT DETERMINISTIC`, a `BEGIN` block containing a `DECLARE` statement for `prezo_total`, a `DECLARE` statement for `continuar`, a `CURSOR FOR` statement, a `SELECT` statement, a `DECLARE CONTINUE HANDLER` statement, a `LOOP` block with `FETCH`, `IF`, `LEAVE`, and `SET` statements, and a `RETURN` statement. The script ends with `END //` and `DELIMITER ;`. The 'Action Output' tab at the bottom shows the execution result: 'CREATE FUNCTION CARLOSFL_prezo_total_pedido(codigo_pedido... 0 row(s) affected' with a duration of 0.0079 sec.

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying a SQL script: `select jardineria.CARLOSFL_prezo_total_pedido(1);`. The 'Result Grid' tab is active, showing the result of the query: a single row with the value 72. The 'Action Output' tab at the bottom shows the execution results for two queries: 'CREATE FUNCTION CARLOSFL_prezo_total_pedido(codigo_pedido... 0 row(s) affected' and 'select jardineria.CARLOSFL_prezo_total_pedido(1) LIMIT 0... 1 row(s) returned' with a duration of 0.0042 sec / 0.0000...

3. Función: `teuser_suma_pedidos_cliente`. Dado un código de cliente a función debe calcular a suma total de todos os pedidos realizados polo cliente. Deberá facer uso da función `calcular_prezo_total_pedido` que desenvolveu no apartado anterior.
- Parámetros de entrada: `codigo_cliente` (INT)



- Parámetros de saída: A suma total de todos os pedidos do cliente (FLOAT)
- Empregar handler para excepcións e warnings.

```
1 DELIMITER //
2
3 CREATE FUNCTION CARLOSFL_suma_pedidos_cliente(codigo_cliente INT)
4 RETURNS FLOAT DETERMINISTIC
5 BEGIN
6 DECLARE total_pedidos FLOAT;
7 DECLARE CONTINUE HANDLER FOR SOLEXCEPTION
8 BEGIN
9 SET total_pedidos = NULL;
10 END;
11
12 SET total_pedidos = 0;
13
14 SELECT SUM(precio_unidad * cantidad) INTO total_pedidos
15 FROM detalle_pedido
16 WHERE codigo_pedido IN (
17 SELECT codigo_pedido
18 FROM pedido
19 WHERE codigo_cliente = codigo_cliente);
20
21 RETURN total_pedidos;
22 END//
23
24 DELIMITER ;
```

#	Time	Action	Message	Duration / Fetch
1	16:39:13	CREATE FUNCTION CARLOSFL_suma_pedidos_cliente(codi...	0 row(s) affected	0.028 sec

Éxito:

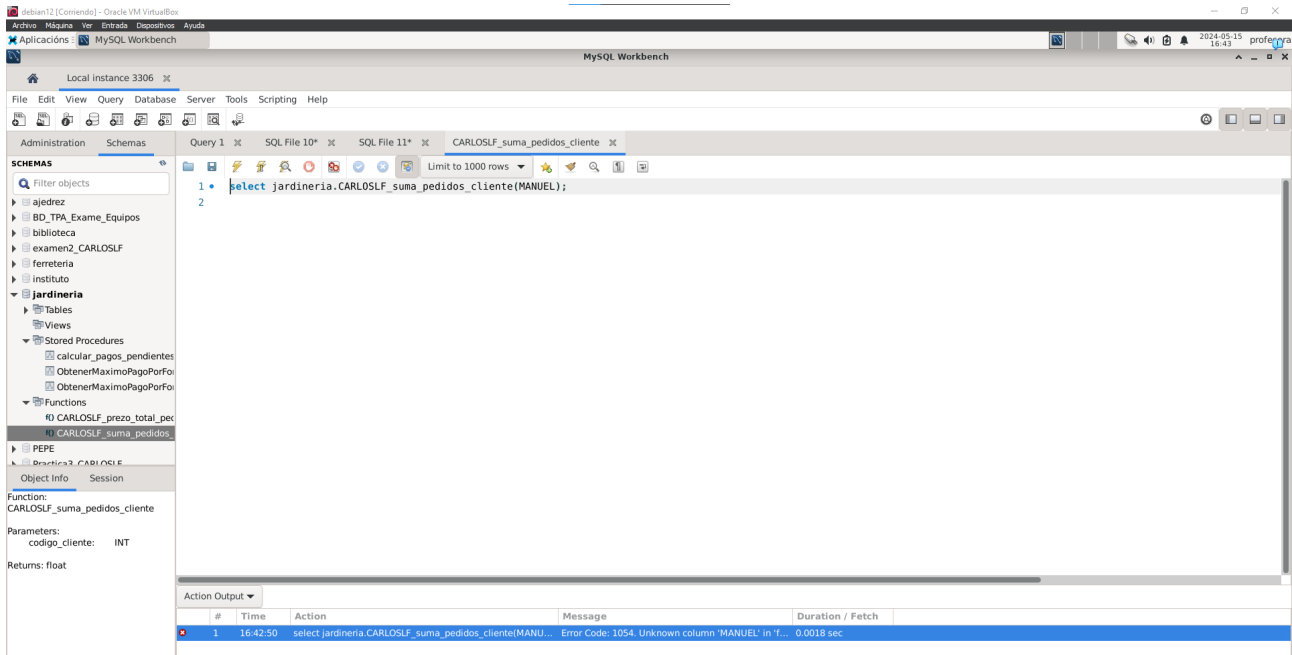
```
1 select jardineria.CARLOSFL_suma_pedidos_cliente(1);
2
```

#	Time	Action	Message	Duration / Fetch
1	16:39:13	CREATE FUNCTION CARLOSFL_suma_pedidos_cliente(codi...	0 row(s) affected	0.028 sec
2	16:39:45	select jardineria.CARLOSFL_suma_pedidos_cliente(1) LIM...	1 row(s) returned	0.027 sec / 0.00004...

#	Time	Action	Message	Duration / Fetch
1	16:39:13	CREATE FUNCTION CARLOSFL_suma_pedidos_cliente(codi...	0 row(s) affected	0.028 sec
2	16:39:45	select jardineria.CARLOSFL_suma_pedidos_cliente(1) LIM...	1 row(s) returned	0.027 sec / 0.00004...



Fallo:



4. Función: teuuser_suma_pagos_cliente. Dado un código de cliente a función debe calcular a suma total dos pagos realizados por ese cliente.
- Parámetros de entrada: codigo_cliente (INT)
 - Parámetros de saída: A suma total de todos os pagos do cliente (FLOAT).
 - Empregar handler para excepcións e warnings.



MySQL Workbench interface showing the creation of a stored procedure named `CARLOSFL_suma_pagos_cliente`. The procedure calculates the total amount paid by a client.

```
1 CREATE DEFINER='root'@'localhost' FUNCTION 'CARLOSFL_suma_pagos_cliente'(codigo_cliente INT) RETURNS float
2 DETERMINISTIC
3 BEGIN
4 DECLARE total_pagos FLOAT;
5 -- Utilizar un controlador de excepciones para manejar errores
6 DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
7 BEGIN
8 -- En caso de excepción, devolver un valor nulo
9 SET total_pagos = NULL;
10 END;
11 -- Inicializar la variable total_pagos
12 SET total_pagos = 0;
13 -- Calcular la suma total de pagos realizados por el cliente
14 SELECT SUM(total) INTO total_pagos
15 FROM pago p
16 WHERE p.codigo_cliente = codigo_cliente;
17 -- Devolver el total de pagos del cliente
18 RETURN total_pagos;
19 END
```

The Action Output table shows the successful execution of the procedure:

#	Time	Action	Message	Duration / Fetch
1	16:25:01	Apply changes to CARLOSFL_suma_pagos_cliente	Changes applied	

MySQL Workbench interface showing the execution of a query that calls the stored procedure `CARLOSFL_suma_pagos_cliente`.

```
1 select jardineria.CARLOSFL_suma_pagos_cliente(1);
2
```

The Result Grid shows the output of the query:

#	jardineria.CARLOSFL_suma_pagos_cliente
1	4000

The Action Output table shows the execution details:

#	Time	Action	Message	Duration / Fetch
1	16:25:47	select jardineria.CARLOSFL_suma_pagos_cliente(1) LIMIT ...	1 row(s) returned	0.0040 sec / 0.0000...
2	16:26:04	SELECT * FROM jardineria.cliente LIMIT 0, 1000	36 row(s) returned	0.0017 sec / 0.0000...



MySQL Workbench interface showing a query execution result.

Query 1: `select jardineria.CARLOSLF_suma_pagos_cliente(2);`

Result Grid:

#	jardineria.CARLOSLF_suma_pago
1	0.0000

Action Output:

#	Time	Action	Message	Duration / Fetch
1	16:26:42	select jardineria.CARLOSLF_suma_pagos_cliente(2) LIMIT ...	1 row(s) returned	0.0022 sec / 0.0000...